第五章

设计模式导论

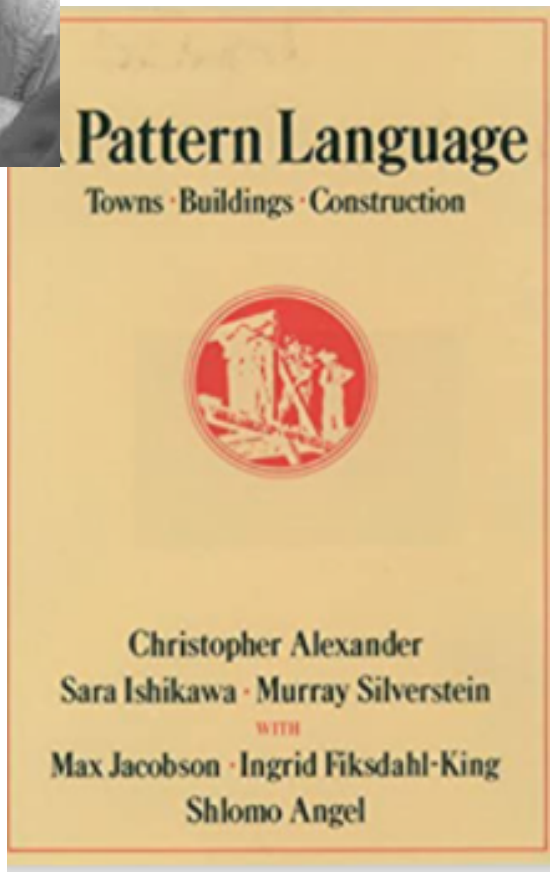
第五章：设计模式导论（上）

- 面向对象设计原则
- 设计模式
- 单例模式

设计模式的发展史

“模式”一词起源于建筑领域





1977年，美国著名建筑大师、加利福尼亚大学伯克利分校环境结构中心主任克里斯托夫·亚历山大（Christopher Alexander）在他的著作《建筑模式语言：城镇、建筑、构造》（A Pattern Language: Towns Building Construction）中描述了一些常见的建筑设计问题，并提出了 253 种关于对城镇、邻里、住宅、花园和房间等进行设计的基本模式。

1979 年他的另一部经典著作《建筑的永恒之道》（The Timeless Way of Building）进一步强化了设计模式的思想，为后来的建筑设计指明了方向。

设计模式的发展史

“每一个模式描述了一个在我们周围不断重复发生的问题，以及该问题的解决方案的核心。这样，你就能一次又一次地使用该方案而不必做重复劳动。”

-- Christopher Alexander



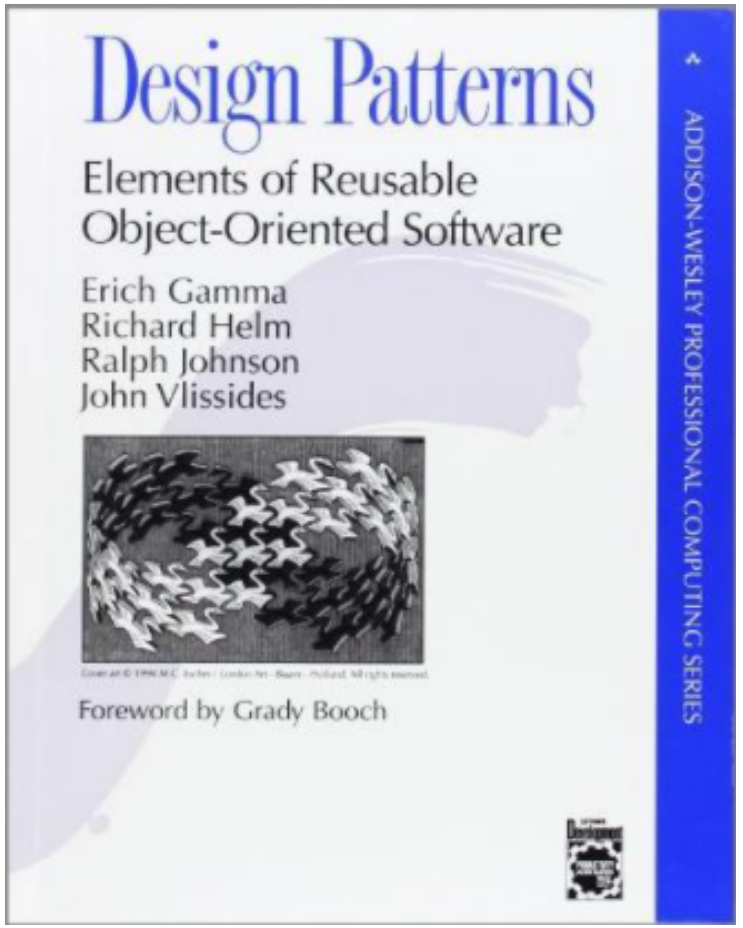
模式的核心：

提供相似问题的解决方案。

设计模式的发展史

“模式”起源于建筑，但同样适用于面向对象的软件设计。在软件开发中，我们用对象和接口代替了墙壁和门窗。

1995 年，由 Erich Gamma、Richard Helm、Ralph Johnson 和 John Vlissides 四人合著出版了一本名为《设计模式 - 可复用面向对象软件的基础 》（ Design Patterns - Elements of Reusable Object-Oriented Software ）的书。



书中收录了 23 个设计模式，首次提到了软件开发中**设计模式**的概念。这是设计模式领域里程碑的事件，导致了软件设计模式的突破。

四位作者合称 **GOF**（四人帮，全拼 Gang of Four ）。

面向对象设计原则

- 在设计面向对象的程序时，需遵循的常用原则有七个，包括：



单一职责原则

- **单一职责原则（Single Responsibility Principle – SRP）定义：**

就一个类而言，应该仅有一个引起它变化的原因。

- 不要把变化原因各不相同的职责放一起，因为不同变化会影响到不相干的职责。
- 如果一个类承担的职责过多，它被复用的可能性越小，就等于把这些职责耦合在一起，而一个职责的变化可能会削弱和抑制这个类完成其他职责的能力。
- 如果多于一个的动机去改变一个类，那么这个类就具有多余一个的职责，就应该要考虑类的职责分离。

开闭原则

开闭原则（Open-Closed Principle – OCP）定义：

一个软件实体应当对扩展开放，对修改关闭。即在设计一个模块的时候，应当使这个模块可以在不被修改的前提下被扩展。

- 通过扩展已有的软件系统，可以提供新的行为，以满足对软件的新需求，使变化中的软件系统有一定的适应性和灵活性。
- 在面向对象编程中，通过抽象类及接口，规定了具体类的特征作为抽象层，相对稳定，不需更改，从而满足“对修改关闭”；而从抽象类导出的具体类可以改变系统的行为，从而满足“对扩展开放”。

依赖倒转原则

- **依赖倒转原理**（Dependency Inversion Principle – DIP）的定义：

高层模块不应该依赖低层模块，他们都应该依赖抽象。要针对接口编程，不要针对实现编程。

- 应当使用接口和抽象类进行变量类型声明、参数类型声明、方法返还类型说明，以及数据类型的转换等，而不要使用具体类。
- 传统的过程性系统的设计办法倾向于使高层次的模块依赖于低层次的模块，抽象层次依赖于具体层次。倒转原则就是把这个错误的依赖关系倒转过来。

里氏代换原则

- **里氏代换原则**（Liskvo Substitution Principle – LSP）的定义：

如果对每一个类型为 T1 的对象 O1 ，都有类型为 T2 的对象 O2 ，使得以 T1 定义的所有程序 P 在所有的对象 O1 都代换为 O2 时，程序 P 的行为没有变化，那么类型 T2 是类型 T1 的子类型。

里氏代换原则

- 一个软件实体如果使用的是一个基类的话，那么一定适用于其子类。而且它觉察不出基类对象和子类对象的区别。也就是说，在软件里面，把基类都替换成它的子类，程序的行为没有变化。
- 反过来的代换不成立，如果一个软件实体使用的是一个子类的话，那么它不一定适用于基类。
- 所有引用基类（父类）的地方必须能透明地使用其子类的对象。

接口隔离原则

- **接口隔离原则**（Interface Segregation Principle – ISP）的定义：

客户端不应该依赖它不需要的接口。

- 使用多个专门的接口比使用单一的总接口要好。也就是说，一个类对另外一个类的依赖性应当是建立在最小的接口上。
- 一旦一个接口太大，则需要将它分割成一些更细小的接口，使用该接口的客户端仅需要知道与之相关的方法即可。

迪米特法则

- 迪米特法则（Law of Demeter – LoD）的定义：

一个软件实体应当尽可能少的与其他实体发生相互作用。

- 每一个软件单位对其他的单位都只有最少的知识，而且局限于那些与本单位密切相关的软件单位。
- 这样，当一个模块修改时，就会尽量少的影响其他的模块。扩展会相对容易。这是对软件实体之间通信的限制。它要求限制软件实体之间通信的宽度和深度。

合成复用原则

- **合成/聚合复用原则**(Composite/Aggregate Reuse Principle – CARP)的定义：尽量使用对象组合，而不是继承来到复用的目的。
  - 在一个新的对象里面使用一些已有的对象，使之成为新对象的一部分；新的对象通过向这些对象的委派达到复用这些对象的目的。
  - 应首先使用合成/聚合，其次才考虑继承。而使用继承时，要严格遵循里氏代换原则。有效地使用继承会有助于对问题的理解，降低复杂度，而滥用继承会增加系统构建、维护时的难度及系统的复杂度。

设计模式的概念

设计模式（Design Pattern）

是一套被反复使用、经过分类编目的代码设计经验的总结。它描述了在软件设计过程中的一些不断重复发生的问题，以及该问题的核心解决方案。其目的是为了**提高代码的可重用性、代码的可阅读性和代码的可靠性**。

设计模式的本质是面向对象设计原则的实际运用，是对类的封装性、继承性和多态性以及类的关联关系和组合关系的充分理解。

设计模式的概念

设计模式的**四个基本要素**

要素	描述
模式名 **Pattern Name**	一个助记词，用一两个词来描述模式的问题、解决方案和效果。
问题 **Problem**	描述了该模式的应用环境，即何时使用该模式。它解释了设计问题和问题存在的前因后果，以及必须满足的一系列先决条件。
解决方案 **Solution**	描述了设计的组成成分、它们之间的相互关系及各自的职责和协作方式。解决方案并不描述一个特定而具体的设计或实现，而是提供设计问题的抽象描述和怎样用一个具有一般意义的元素组合（类或对象的组合）来解决这个问题。
效果 **Consequence**	描述了模式的应用效果以及使用该模式应该权衡的问题，即模式的优缺点。主要是对时间和空间的衡量，以及该模式对系统的灵活性、扩充性、可移植性的影响等。

设计模式的作用

- 重用设计，重用设计比重用代码更有意义，它会自动带来代码的重用。
- 为设计提供共同的词汇，每个模式名就是一个设计词汇，其概念是的程序员间的交流更加方便。
- 在开发文档中采用模式词汇可以让其他人更容易理解你的想法和做法，编写开发文档也更方便。
- 应用设计模式可以让重构系统变得容易，可以确保开发正确的代码，并降低在设计或实现中出现错误的可能。
- 支持变化，可以为重写其他应用程序提供很好的系统架构。
- 正确使用设计模式，可以节省大量时间。

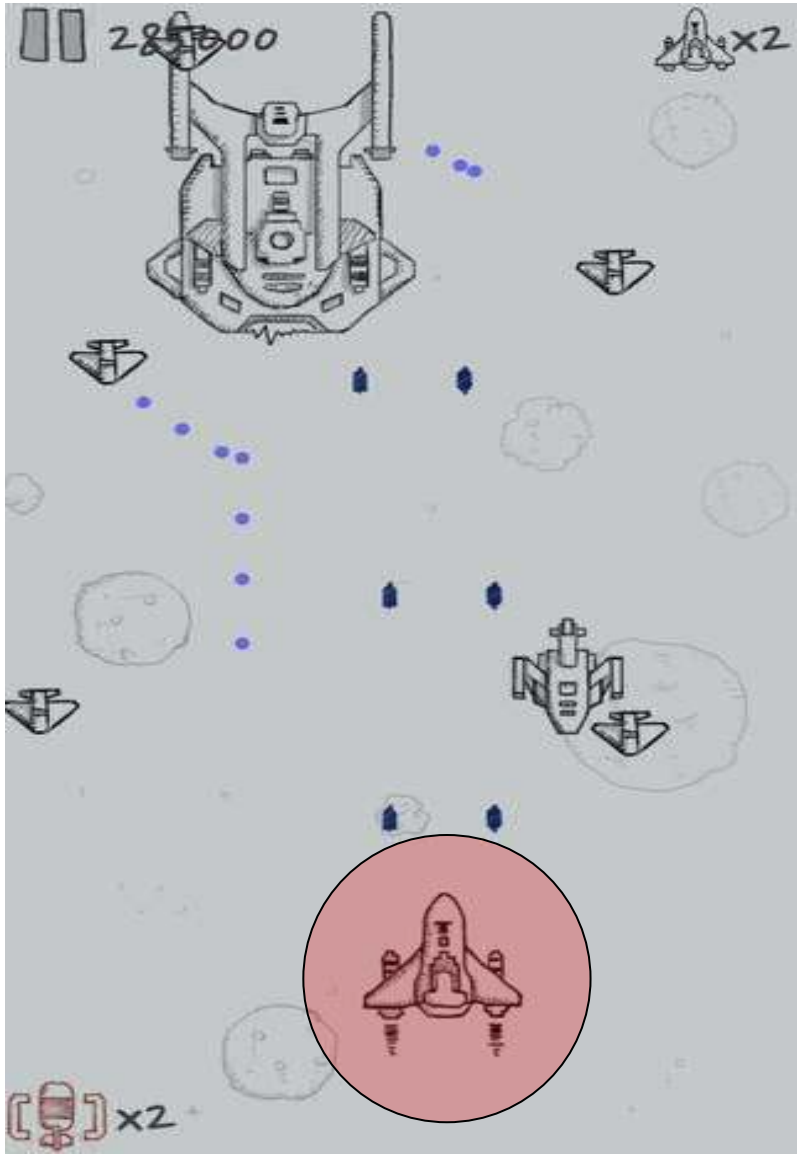
设计模式的作用

- 根据**目的**来分（模式是用来完成什么工作）
  - 创建型模式：用于描述“怎样创建对象”，它的主要特点是“将对象的创建与使用分离”。
  - 结构型模式：用于描述如何将类或对象按某种布局组成更大的结构行。
  - 行为型模式：用于描述类或对象之间怎样相互协作共同完成单个对象都无法单独完成的任务，以及怎样分配职责。
- 根据**范围**来分（模式主要用于类还是对象）
  - 类模式：用于处理类与子类之间的关系，这些关系通过继承来建立，是静态的，在编译时刻便确定下来了。
  - 对象模式：用于处理对象之间的关系，这些关系可以通过组合或聚合来实现，在运行时刻是可以变化的，更具动态性。

设计模式的分类

GoF 23种设计模式的分类图:

		目的		
		创建型	结构型	行为型
范围	类	工厂方法	适配器（类）	模板方法、解释器
	对象	抽象工厂 生成器 原型 单例	适配器（对象） 桥接 组合 装饰 外观 享元 代理	职责链 命令 迭代器 中介者 备忘录 观察者 状态 策略 访问者



单例模式

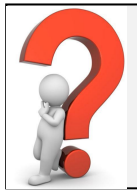
单例模式

- **单例模式（Singleton Pattern）**

保证一个类仅有一个实例，并提供一个访问它的全局访问点。

- 很多时候我们都要保证一个类只有一个实例
  - 整个应用程序中， 只有一个连接数据库的connection实例
  - 操作系统只有一个时钟
  - **飞机大战游戏中只有一个英雄机**
  - **…**

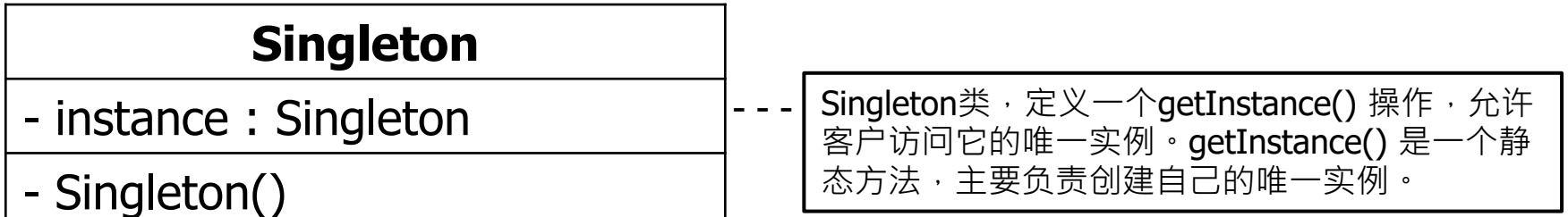
单例模式



如何保证一个类只有一个实例，且这个实例易于被访问呢？

全局变量使得一个对象可以被全局访问，但它不能防止你实例化多个对象。一个最好的办法就是让类自身负责保存它的唯一实例，这个类可以保证没有其他实例可以被创建，并且它可以提供一个访问该实例的方法。

单例模式（Singleton）结构图



单例模式（1）：饿汉式

经典单例模式实现

Singleton类，定义一个getInstance() 操作，允许客户访问它的唯一实例。getInstance() 是一个静态方法，主要负责创建自己的唯一实例。

```
public class Singleton{
    private static Singleton singleton = new Singleton ();

    private Singleton(){
    }

    public static Singleton GetInstance () {
      return singleton;
    }
}
```

1.**类初始化时，立即加载这个对象**，天然的线性安全保障

2. 把构造方法声明为私有，使外界无法利用new创建此类实例

3.方法无需同步，调用效率高

单例模式（2）：懒汉式

经典单例模式实现

Singleton类，定义一个getInstance() 操作，允许客户访问它的唯一实例。getInstance() 是一个静态方法，主要负责创建自己的唯一实例。

```
public class Singleton {
    private static Singleton instance;
    private Singleton (){}

    public static Singleton getInstance() {
    if (instance == null) {
        instance = new Singleton();
    }
    return instance;
    }
}
```

1. 利用一个私有的静态变量来记录Singleton类的唯一实例

2. 把构造方法声明为私有，使外界无法利用new创建此类实例

3. 利用公有的静态方法getInstance()来实例化对象，并返回该唯一实例

单例模式（3）：同步锁



在懒汉式里多线程同时访问Singleton类 ， 调用getInstance()方法，则有可能创建多个实例，如何避免？

通过增加**synchronized**关键字到 getInstance() 方法中，迫使每个线程在进入这个方法之前， 要先等候别的线程离开该方法。也就是说， 不会有两个线程可以同时进入这个方法。

```
public class Singleton {
    private static Singleton instance;
    private Singleton (){}
    public static synchronized Singleton getInstance() {
    if (instance == null) {
        instance = new Singleton();
    }
    return instance;
    }
}
```

线程安全， 但效率低！

课外拓展：双重检验锁

单例模式

单例模式总结：

要素	描述
模式名 **Pattern Name**	单例模式（**Singleton Pattern**）
目的 **Intent**	只希望有一个对象，系统所有地方都可以访问这个对象，且不使用全局变量，也不需要传递对象的引用。
问题 **Problem**	几个不同的客户对象都希望引用同一个对象，如何保证？
解决方案 **Solution**	保证一个唯一实例 1）定义私有的静态成员变量保存单实例的引用；2）定义公有的静态方法getInstance() 获取唯一实例；3）该类自己负责"第一次使用时"实例化对象。
效果 **Consequence**	优点：1）对唯一实例的受控访问；2）在内存里只有一个实例，减少了内存的开销，避免频繁地创建销毁对象，提高性能；3）避免对共享资源的多重占用；4）允许可变数目的实例；5）可以全局访问。

第五章
设计模式导论（下）

第五章：设计模式导论（下）

- 简单工厂模式
- 工厂模式
- 抽象工厂设计模式

简单工厂及工厂模式

案例背景

- 我们在大学城食堂开了家简单的披萨店，向附近的学生提供多种口味的比萨：
  - 芝士比萨（CheesePizza）
  - 希腊比萨（GreekPizza）
  - 腊肠比萨（PepperoniPizza）

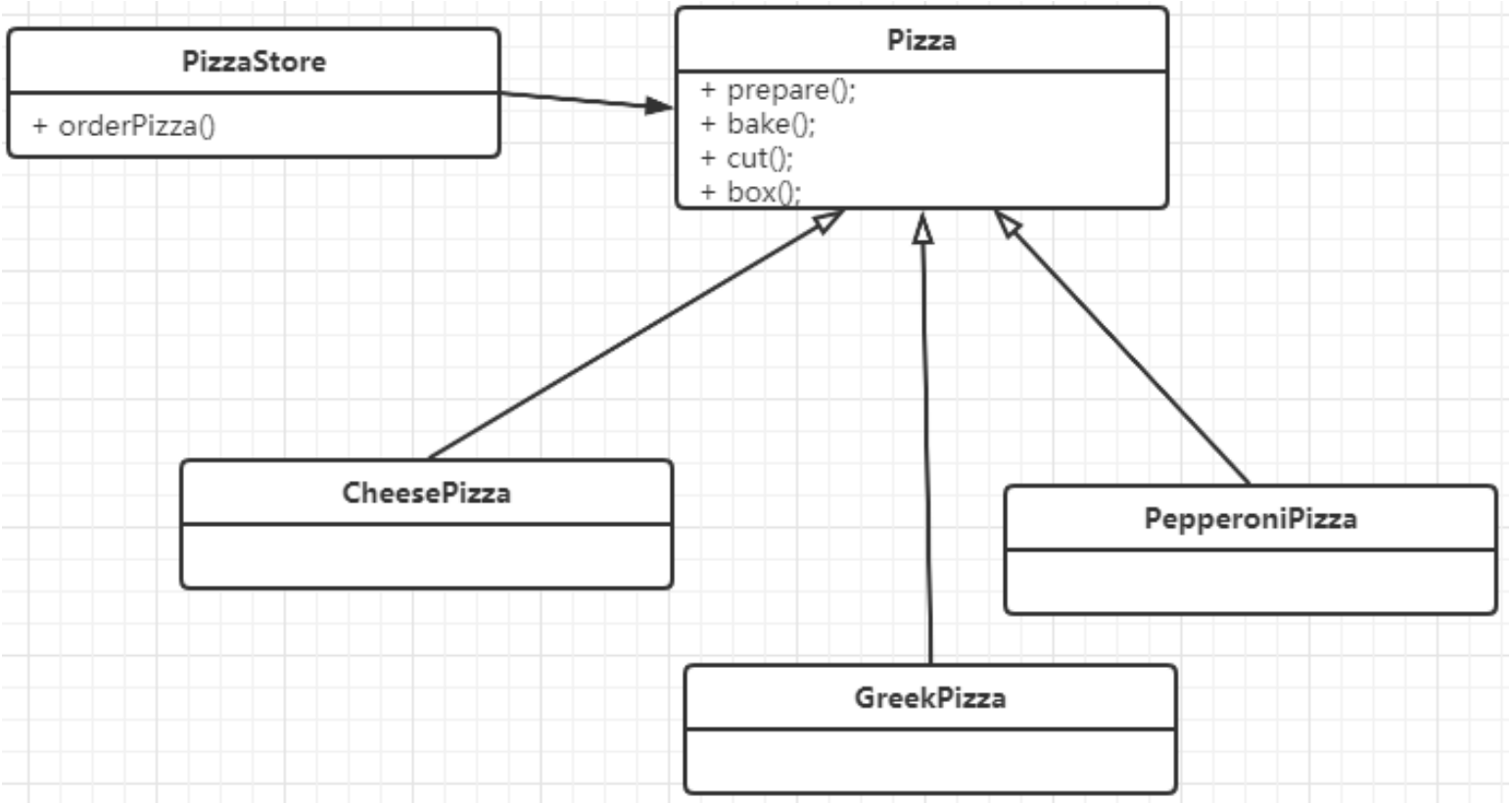




- 请设计一个模拟的比萨店
  - 软件系统

案例背景

设计UML图：



为了让系统更有弹性，我们希望Pizza是一个抽象类或接口，可以利用面向对象的继承机制。

案例背景

代码实现：

```
Pizza orderPizza(String type) {
    Pizza pizza;

    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    }else if (type.equals("greek")) {
        pizza = new GreekPizza();
    }else if (type.equals("pepperoni")) {
        pizza = new PepperoniPizza();
    }

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();

    return pizza;
}
```

通过参数传入Pizza类型

根据pizza类型，实例化正确的具体类，然后将其赋值给Pizza实例变量。

一旦我们获取了一个具体的Pizza类的实例，就可以调用它的方法来来做一些准备，然后烘烤、切片、装盒。

1. 符合开闭原则吗？
2. 符合单一职责原则吗？

案例背景



根据市场变化，如何增加新口味的比萨或下架不受客户欢迎的比萨？在前面的设计中，哪些是不变化的？哪些可能是变化的？

```
Pizza orderPizza(String type) {
    Pizza pizza;

    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    }else if (type.equals("greek")) {
        pizza = new GreekPizza();
    }else if (type.equals("pepperoni")) {
        pizza = new PepperoniPizza();
    }else if (type.equals("clam")) {
        pizza = new ClamPizza();
    }else if (type.equals("veggie")) {
        pizza = new VeggiePizza();
    }

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();

    return pizza;
}
```

这是**变化**的部分，代码随着比萨菜单的改变而变化，没有对修改封闭，违反OCP原则（开闭原则）。

这是**不变化**的部分，代码保持不变。

案例背景

修改设计，封闭创建对象的代码：

- 将创建比萨对象的代码从orderPizza()方法中抽离
- 定义一个新的类SimplePizzaFactory，专门负责创建比萨对象

```
public class SimplePizzaFactory{
    public Pizza createPizza(String type) {
        Pizza pizza = null;

        if (type.equals("cheese")) {
            pizza = new CheesePizza();
        }else if (type.equals("greek")) {
            pizza = new GreekPizza();
        }else if (type.equals("pepperoni")) {
            pizza = new PepperoniPizza();
        }else if (type.equals("clam")) {
            pizza = new ClamPizza();
        }else if (type.equals("veggie")) {
            pizza = new VeggiePizza();
        }

        return pizza;
    }
}
```

定义createPizza()方法，使用这个方法来实例化新对象。

这是从orderPizza() 方法中移出来的代码，代码没有什么变动。

案例背景

修改设计，封闭创建对象的代码：

- 重做PizzaStore类

如果想把这个地方删除，应该怎么做？

```
public class PizzaStore {
    SimplePizzaFactory factory;

    public PizzaStore(SimplePizzaFactory factory) {
        this.factory = factory;
    }

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = factory.createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
    }
}
```

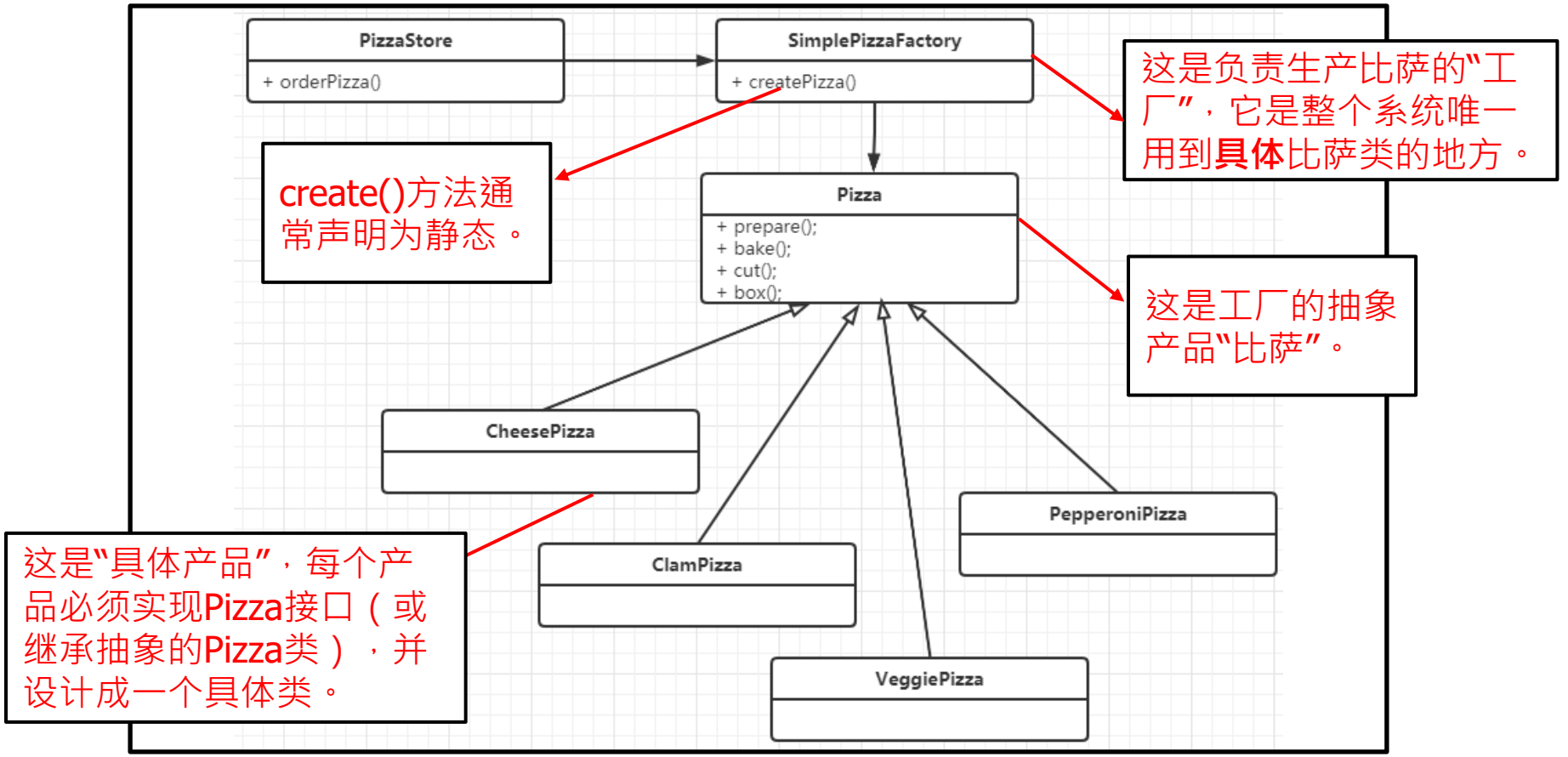
PizzaSore的**构造器**，需要一个工厂作为参数。

orderPizza() 方法通过简单传入订单类型来使用工厂创建比萨。

注意：我们把new操作符替换成工厂对象的创建方法。这里不再使用具体实例化。

简单工厂模式

简单工厂模式 (SimpleFactory Pattern) 并不是一个真正意义上设计模式，但工厂模式是建立在它之上。



互动小问题

简单工厂模式 (SimpleFactory Pattern)是否满足开闭原则呢？

互动小问题

简单工厂模式 (SimpleFactory Pattern)似乎只是把问题搬到另一个对象罢了，问题依然存在，是吗？

工厂方法模式

需求继续变化…

- 比萨店经营有成，接下来的计划就是在多个不同城市开设加盟店
  - 质量控制：确保每个加盟店营运质量
    - ✓ 要求每个加盟店采用统一的标准方式制作比萨
    - ✓ （准备、烘烤、切片、装盒）
  - 区域差异：每个加盟店需要适应当地口味
    - ✓ 纽约（NY）比萨工厂，生产纽约风味的各种比萨
    - ✓ 芝加哥（Chicago）比萨工厂，生产芝加哥风味的各种比萨

结论：需要建立一个框架，把PizzaStore和创建Pizza捆绑在一起， 同时又保持一定的弹性。

工厂方法模式



如何让披萨制作活动局限于PizzaStore类，而同时又能让这些加盟店可以自由地制作该地区的风味？

把createPizza()方法放回到PizzaStore中， 并设置为抽象方法，然后为每个区域创建一个PizzaStore的子类。

```
public abstract class PizzaStore {

    public Pizza orderPizza(String type) {

        Pizza pizza;

        pizza = createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box;

        return pizza;
    }

    abstract Pizza createPizza(String type);
}
```

createPizza()方法从工厂对象移回到PizzaStore。

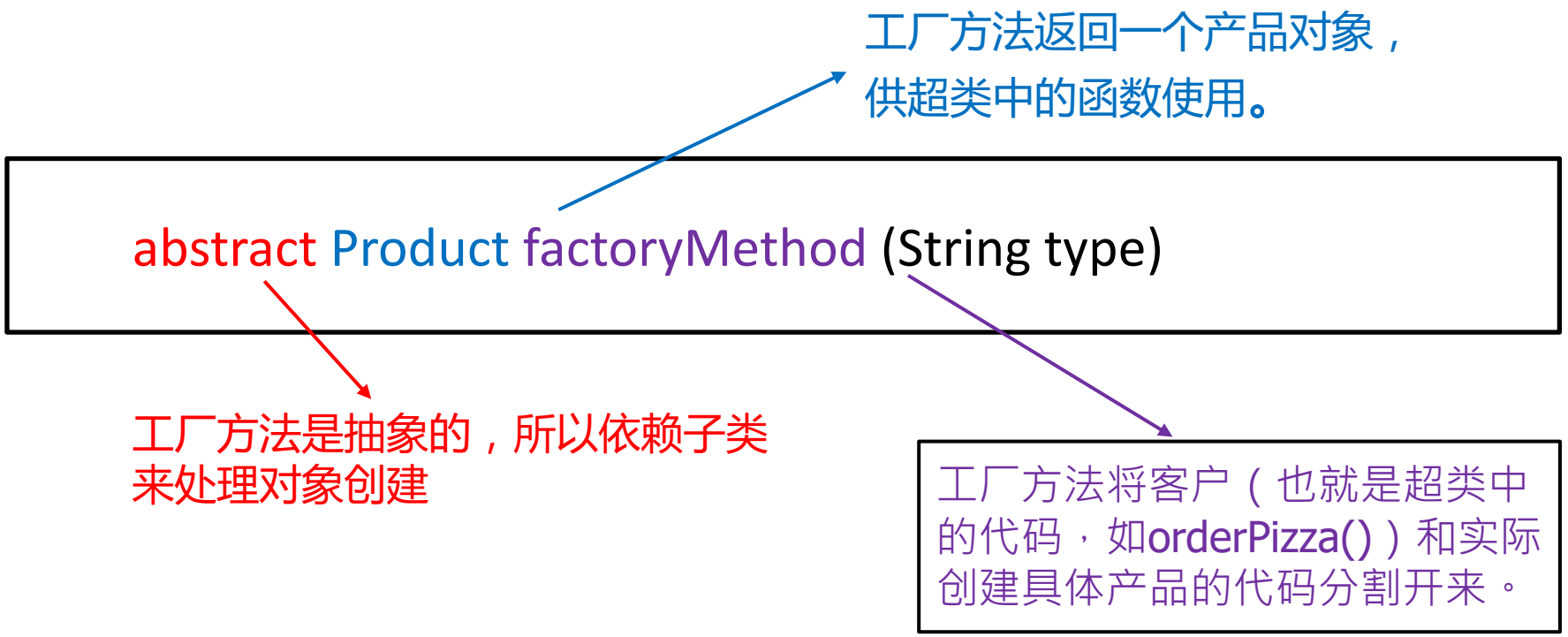
这些功能仍然不变

这个抽象方法又叫做工厂方法，位于PizzaStore。

工厂方法模式

工厂方法（Factory Method）

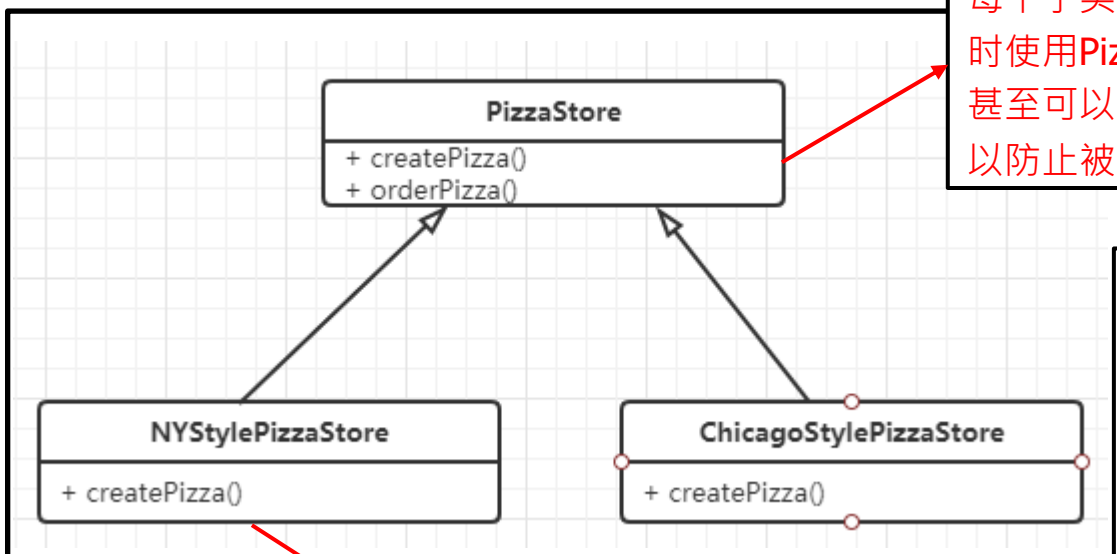
用来处理对象的创建，并将这样的行为封装在子类中。这样，客户程序中关于超类的代码就和子类对象创建代码解耦了。



工厂方法模式



如何让披萨制作活动局限于PizzaStore类，而同时又能让这些加盟店可以自由地制作该地区的风味？



每个子类都会覆盖createPizza()方法，同时使用PizzaStore定义的orderPizza()方法。甚至可以把orderPizza()方法声明为final，以防止被子类覆盖。

orderPizza()方法定义在**抽象**的PizzaStore类内，而不是在子类中。它通过调用createPizza()取得比萨对象，但它并不知道实际是哪个比萨店子类来制作比萨，它只负责后续的准备、烘烤等工作。比萨的类型是由具体的比萨店子类来决定。

这就是"解耦"(decouple)！

如果加盟店要为顾客提供纽约风味的比萨，就使用NYStylePizzaStore子类。因为此类的createPizza()方法会创建纽约风味的比萨。

工厂方法模式

比萨店子类（纽约分店）的实现：

NYStyleCheesePizza 从何而来？

```
public class NYPizzaStore extends PizzaStore {

    @Override
    Pizza createPizza(String type) {
        Pizza pizza = null;

        if (type.equals("cheese")) {
            pizza = new NYStyleCheesePizza();
        }else if (type.equals("pepperoni")) {
            pizza = new NYStylePepperoniPizza();
        }else if (type.equals("clam")) {
            pizza = new NYStyleClamPizza();
        }else if (type.equals("veggie")) {
            pizza = new NYStyleVeggiePizza();
        }

        return pizza;
    }
}
```

NYPizzaStore继承PizzaStore，所以拥有orderPizza()方法。

createPizza()方法返回一个抽象的Pizza对象，由子类全权负责该实例化哪一个具体类型的Pizza。

子类必须实现createPizza()方法，创建具体的比萨类。对于每一种披萨，都是创建纽约风味。

注意：超类PizzaStore的orderPizza()方法并不知道正在创建的比萨是哪一种，它只知道这个比萨可以被准备、烘烤、切片、装盒！

工厂方法模式

比萨超类的实现：

```
public abstract class Pizza {

    String name;
    String dough;
    String sauce;
    ArrayList toppings = new ArrayList();

    void prepare() {
        System.out.println("Preparing: " + name);
        System.out.println("Tossing dough...");
        System.out.println("Adding sauce...");
        System.out.println("Adding toppings: ");
        for(int i=0;i<toppings.size();i++) {
            System.out.println("   "+toppings.get(i));
        }
    }

    void bake() {
        System.out.println("Bake for 25 minutes at 350");
    }

    void cut() {
        System.out.println("Cutting the pizza into diagonal slices");
    }

    void box() {
        System.out.println("Place pizza in official PizzaStore box");
    }

    public String getName() {
        return name;
    }
}
```

每个比萨都具有名称、面团类型、酱料类型和一套佐料。

准备工作需要以特定的顺序进行，有一连串的步骤。

Pizza 抽象类提供了某些默认的基本做法，用来进行烘烤、切片、装盒。

工厂方法模式

比萨子类的实现：

```
public class NYStyleCheesePizza extends Pizza{
    public NYStyleCheesePizza() {
        name = "NY Style Sauce and Cheese Pizza";
        dough = "Thin Crust Dough"
        sauce = "Marinara Sauce"
        toppings.add("Grated Reggiano Cheese);
    }
}
```

纽约风味的比萨有自己的大蒜番茄酱和薄饼。上面覆盖意大利reggiano高级干酪。

```
public class ChicagoStyleCheesePizza extends Pizza {
    public ChicagoStyleCheesePizza() {
        name = "Chicago Style Deep Dish Cheese Pizza";
        dough = "Extra Thick Crust Dough"
        sauce = "Plum Tomato Sauce"
        toppings.add("Shredded Mozzarella Cheese");
    }
}
```

芝加哥风味的比萨使用小番茄作为酱料，并使用厚饼。上面覆盖意大利白干酪。

订购比萨

```
public class PizzatestDrive {

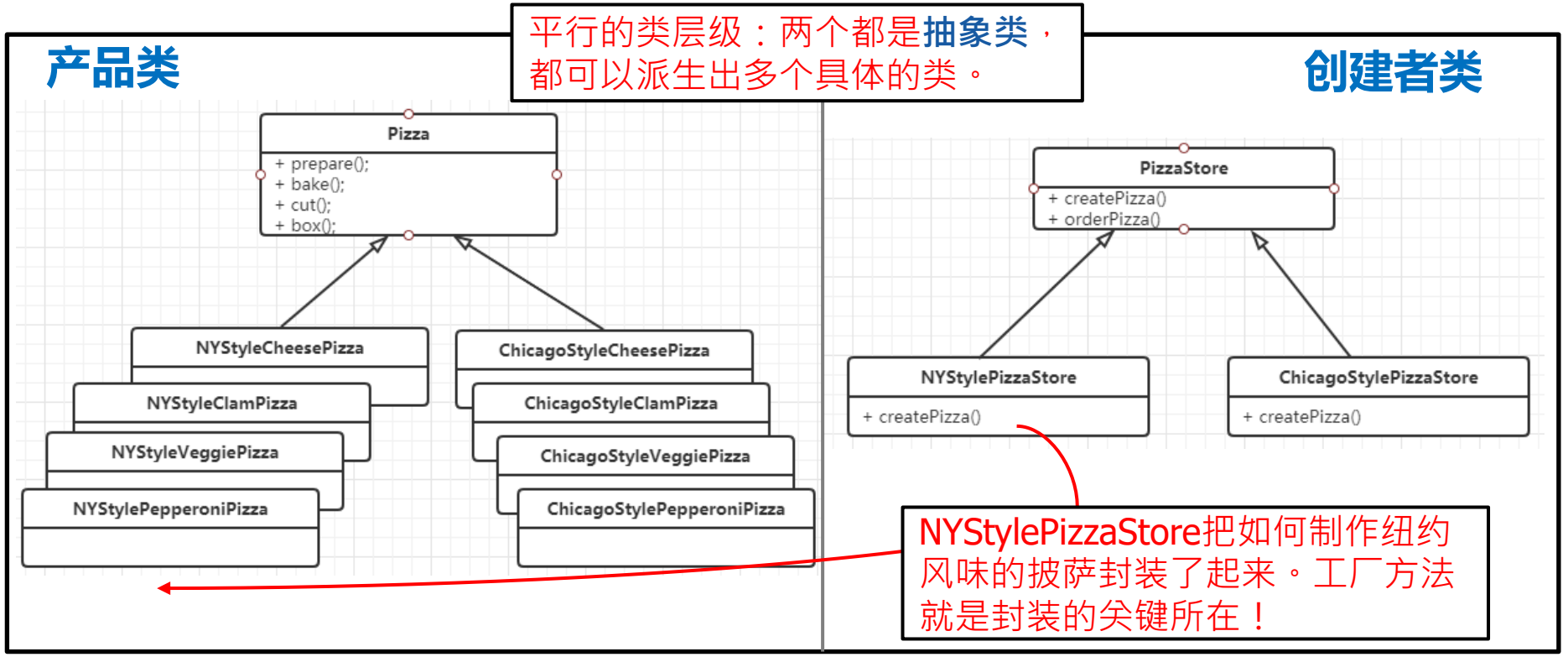
    public static void main(String[] args) {
        PizzaStore nyStore = new NYStylePizzaStore();
        PizzaStore chicagoStore = new ChicagoStylePizzaStore();

        Pizza pizza = nyStore.orderPizza("cheese");
        System.out.println("Ethan ordered a " + pizza.getName() +"\n");

        pizza = chicagoStore.orderPizza("cheese");
        System.out.println("Joel ordered a " + pizza.getName() +"\n");
    }
}
```

工厂方法模式

修改设计，封装具体类型的实例化：

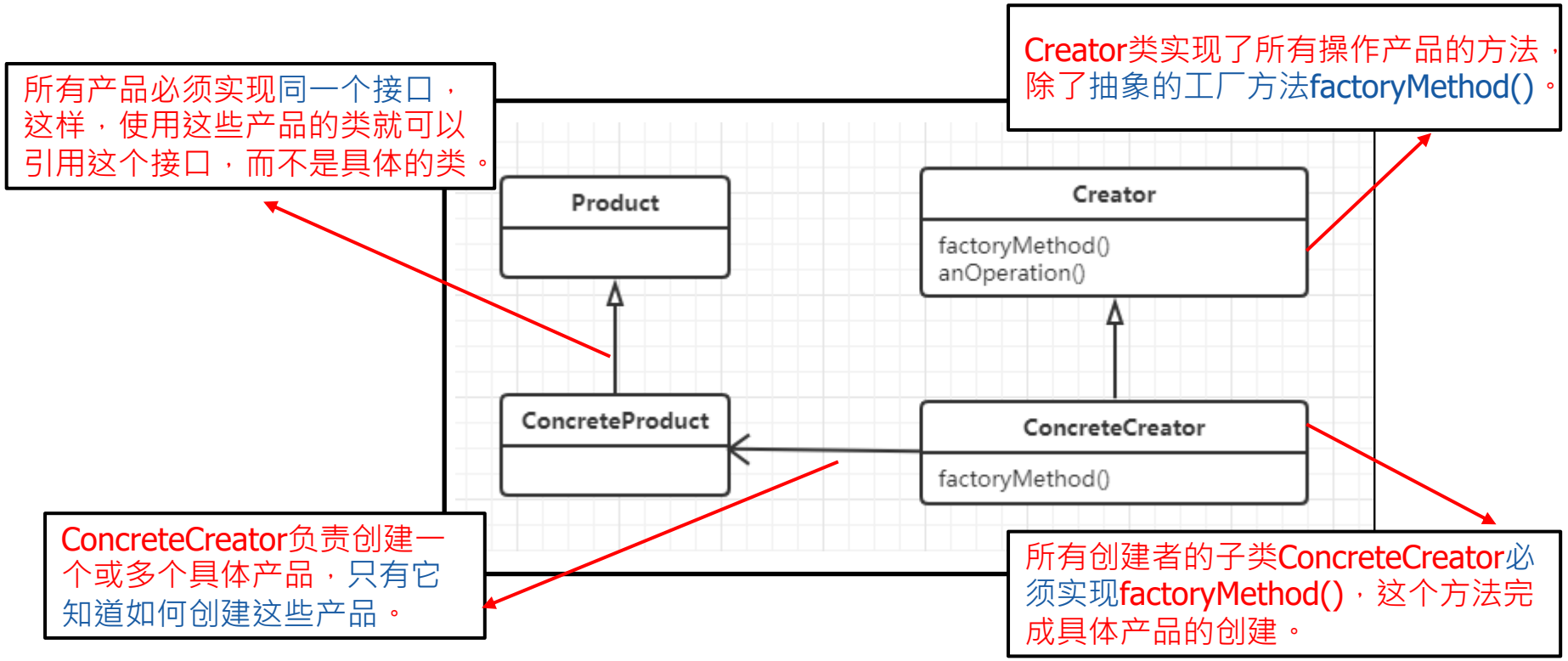


框架结构：将orderPizza()方法和一个工厂方法联合起来，并且工厂方法将产品制作的细节封装进各个创建者。

工厂方法模式

- **工厂方法模式（Factory Method Pattern）**

定义了一个创建对象的接口，让子类决定实例化哪一个类。工厂方法使一个类的实例化延迟到其子类。



简单工厂模式 VS 工厂方法模式

- 简单工厂模式
  - 创建对象的逻辑判断放在了工厂类中，客户不感知具体的类，但是其违背了开闭原则，如果要增加新的披萨类型，就必须修改工厂类。

- 工厂方法模式
  - 通过扩展来新增具体类的，符合开闭原则，但是在客户端就必须要感知到具体的工厂类，也就是将判断逻辑由简单工厂的工厂类挪到客户端。
  - 工厂模式横向扩展很方便，假如又有新区域的加盟店，那么只需要创建相应的披萨店子类和披萨子类去实现抽象工厂接口和抽象产品接口即可，而不用去修改原有已经存在的代码。

互动小问题

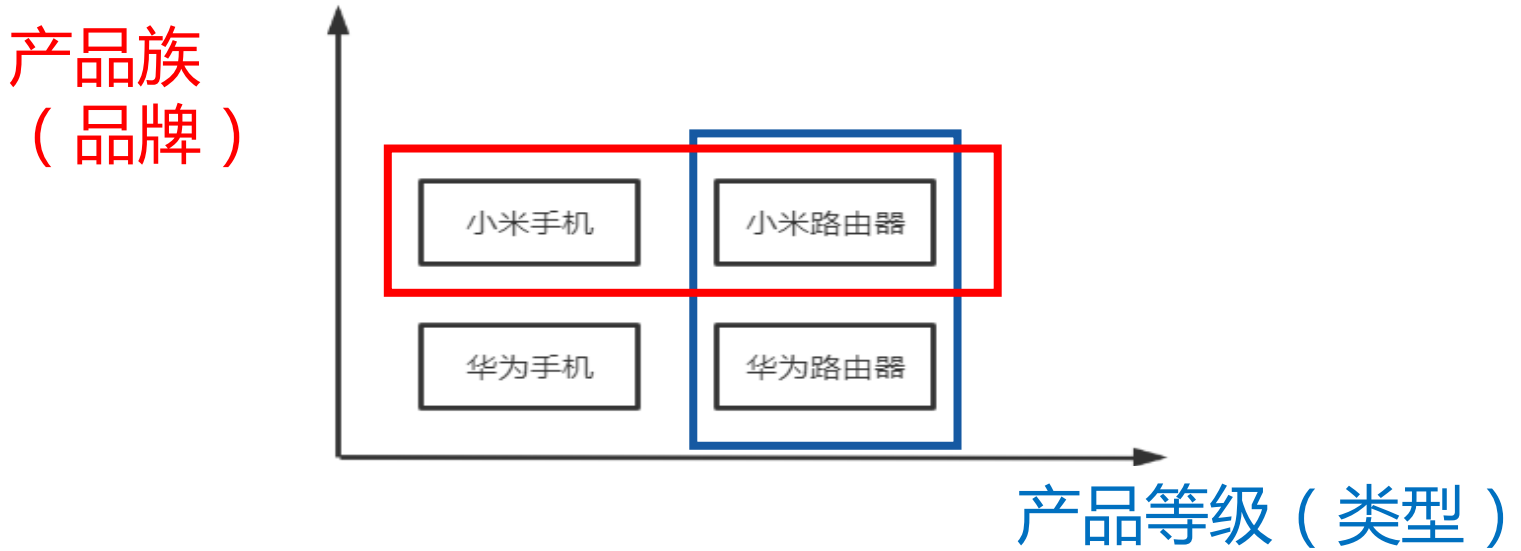
工厂模式 (SimpleFactory Pattern)中某个加盟店（NYPizzaStore）想新增加一个口味怎么办？

抽象工厂模式

案例背景

- **理解产品族和产品等级**

  - 产品族（品牌）：一个品牌下面的所有产品，例如华为下面的手机，路由器，电脑称为华为的产品族。
  - 产品等级（类型）：多个品牌下面的同种产品，例如华为和小米下面的手机称为一个产品。



抽象工厂模式概述

- 抽象工厂模式（Abstract Factory Pattern）是围绕一个超级工厂创建其他工厂。该超级工厂又称为其他工厂的工厂。这种类型的设计模式属于创建型模式，它提供了一种创建对象的最佳方式。

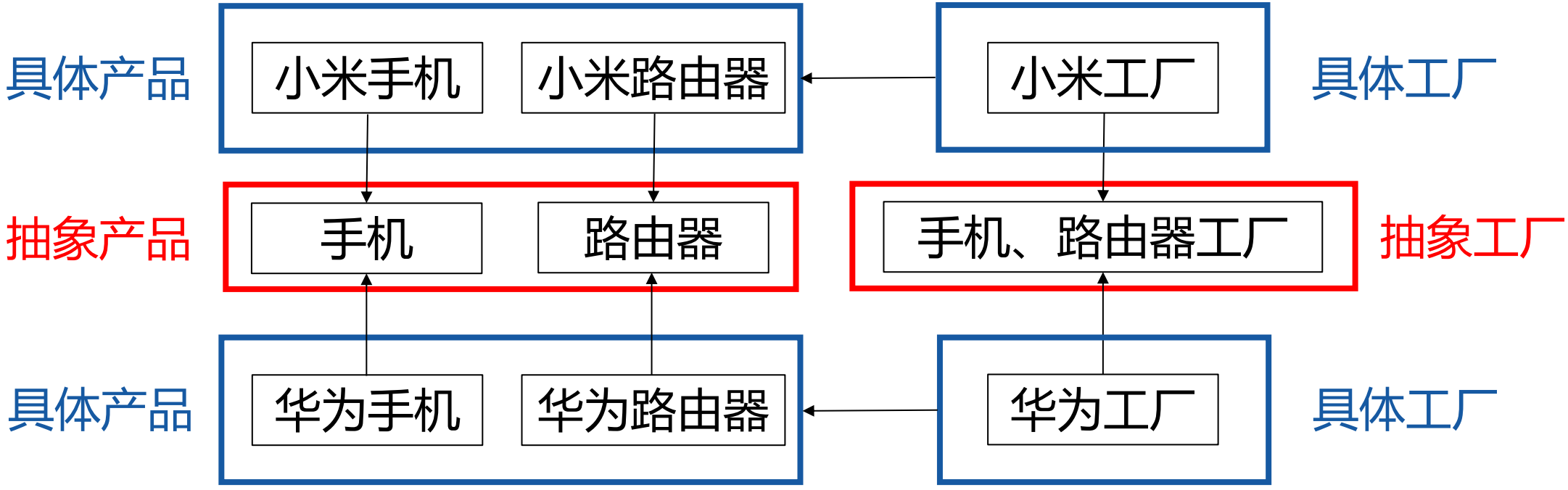
案例背景

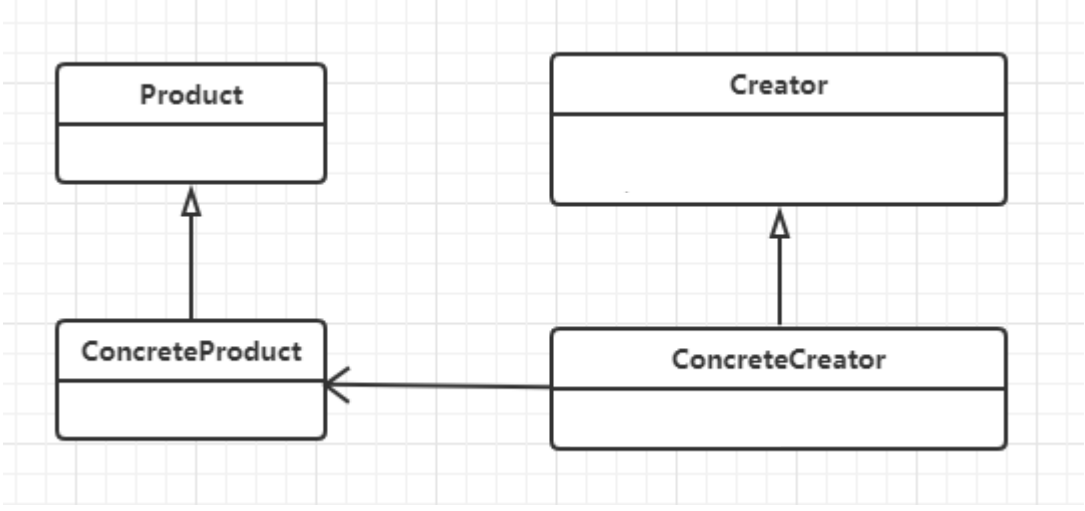
- 有手机和路由器两种产品，有华为和小米两种品牌，两种品牌都可以生产手机和路由器；
- 有手机和路由器两种产品，定义2个接口；
- 小米和华为都可以生产这两种产品，所以有4个实现类；
- 现在需要创建华为和小米的工厂类，先将工厂类进行抽象，里面有创建两个产品的方法，返回的是产品的接口类；
- 创建华为和小米的工厂实现类，继承工厂类接口，实现创建各自产品的方法；
- 客户端调用时，直接用工厂接口类创建需要的工厂，拿到对应的产品。





案例背景


具体产品
小米手机
小米路由器
小米工厂
具体工厂
抽象产品
手机
路由器
手机、路由器工厂
抽象工厂
具体产品
华为手机
华为路由器
华为工厂
具体工厂


Product
Creator
ConcreteProduct
ConcreteCreator

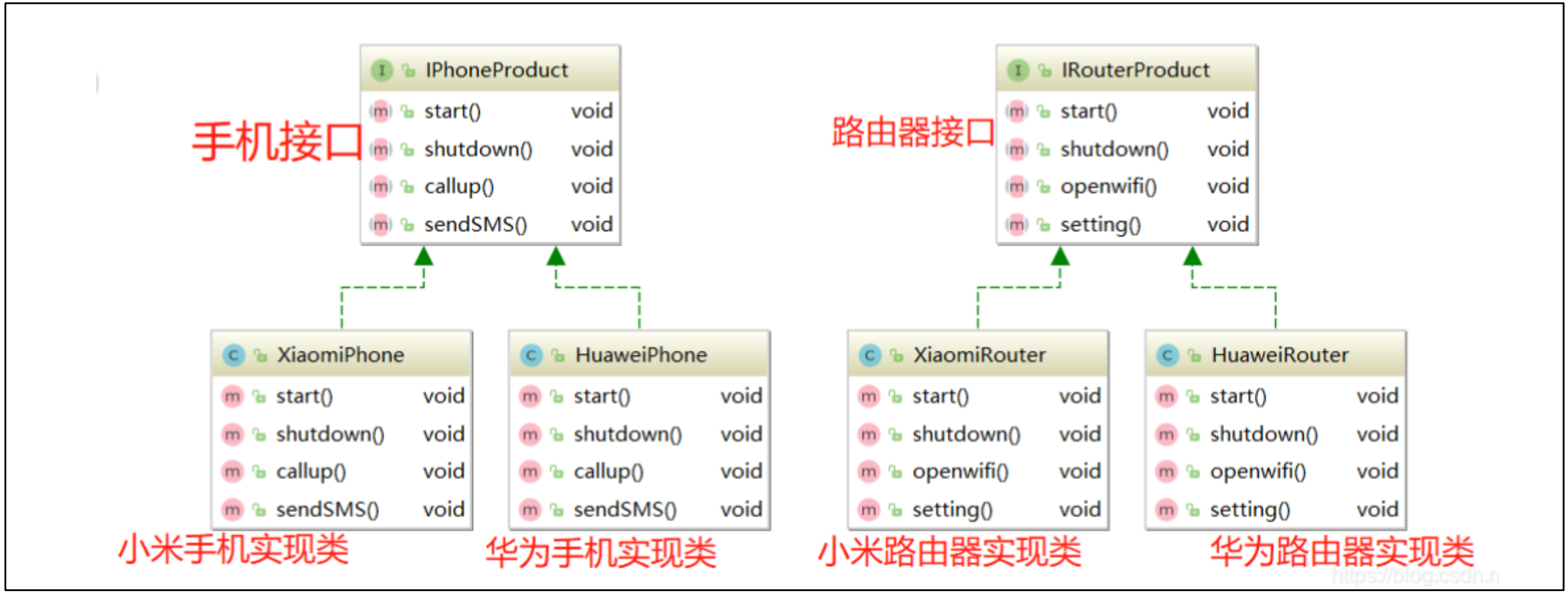
抽象工厂模式

类图分析：


工厂接口类，生产手机和路由器
IProductFactory
phoneProduct()
IPhoneProduct
routerProduct()
IRouterProduct
华为工厂，生产华为手机和华为路由器
小米工厂，生产小米手机和小米路由器
HuaweiFactory
phoneProduct()
IPhoneProduct
routerProduct()
IRouterProduct
XiaomiFactory
phoneProduct()
IPhoneProduct
routerProduct()
IRouterProduct

抽象工厂模式

类图分析：


IPhoneProduct
start() void
shutdown() void
callup() void
sendSMS() void
手机接口
IRouterProduct
start() void
shutdown() void
openwifi() void
setting() void
路由器接口
XiaomiPhone
start() void
shutdown() void
callup() void
sendSMS() void
小米手机实现类
HuaweiPhone
start() void
shutdown() void
callup() void
sendSMS() void
华为手机实现类
XiaomiRouter
start() void
shutdown() void
openwifi() void
setting() void
小米路由器实现类
HuaweiRouter
start() void
shutdown() void
openwifi() void
setting() void
华为路由器实现类

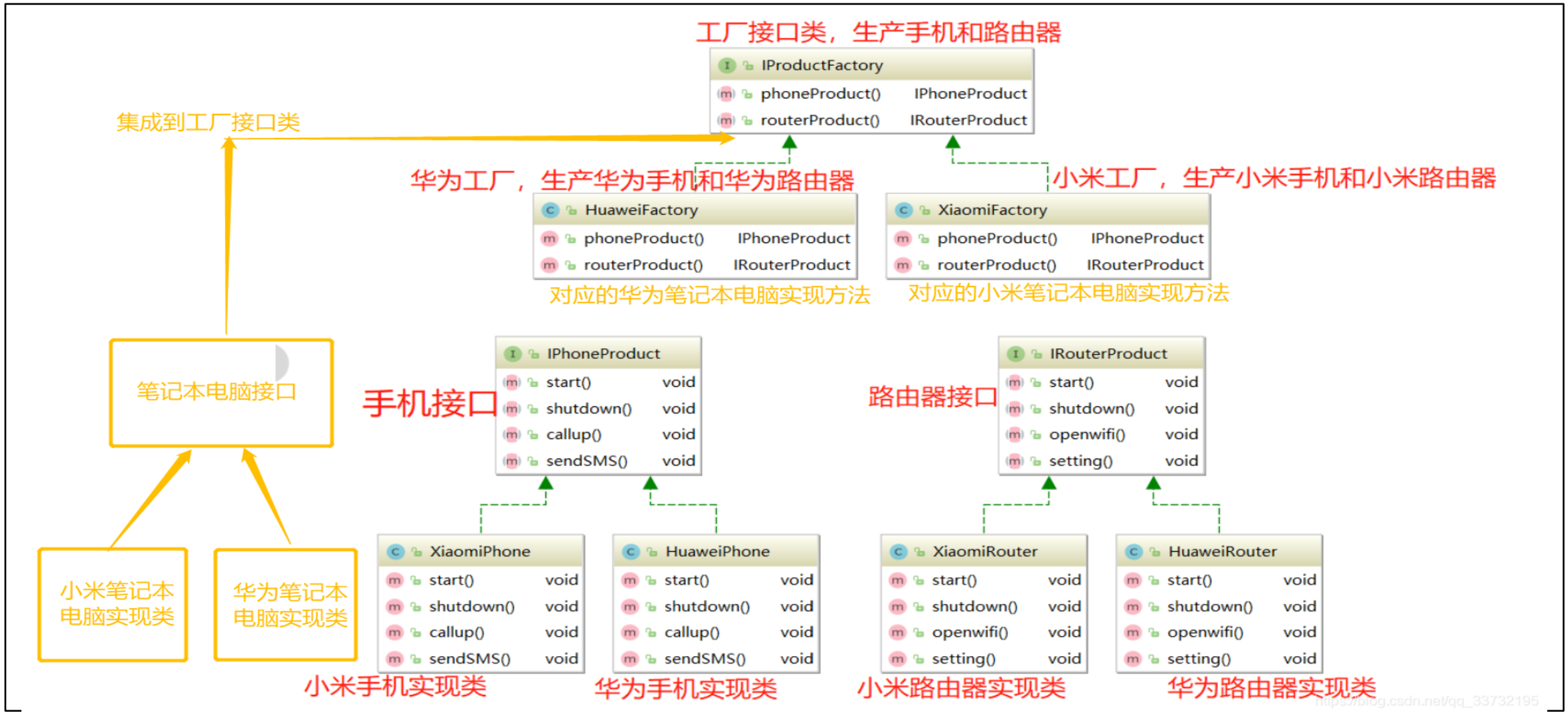
抽象工厂模式

以上就是抽象工厂模式的实现，现在回看产品族和产品等级的概念，
如果新增一个品牌产品族或产品类型会怎样？

拓展一个产品类型

- 我们会发现，拓展一个产品类型是非常困难的，例如产品类型中新增笔记本电脑，也就是说华为和小米现在可以生产电脑了，如下页图片所示（黄色字体为新增一个产品类型需要做的事），对顶层的工厂接口类也要修改，这是非常麻烦的。

抽象工厂模式

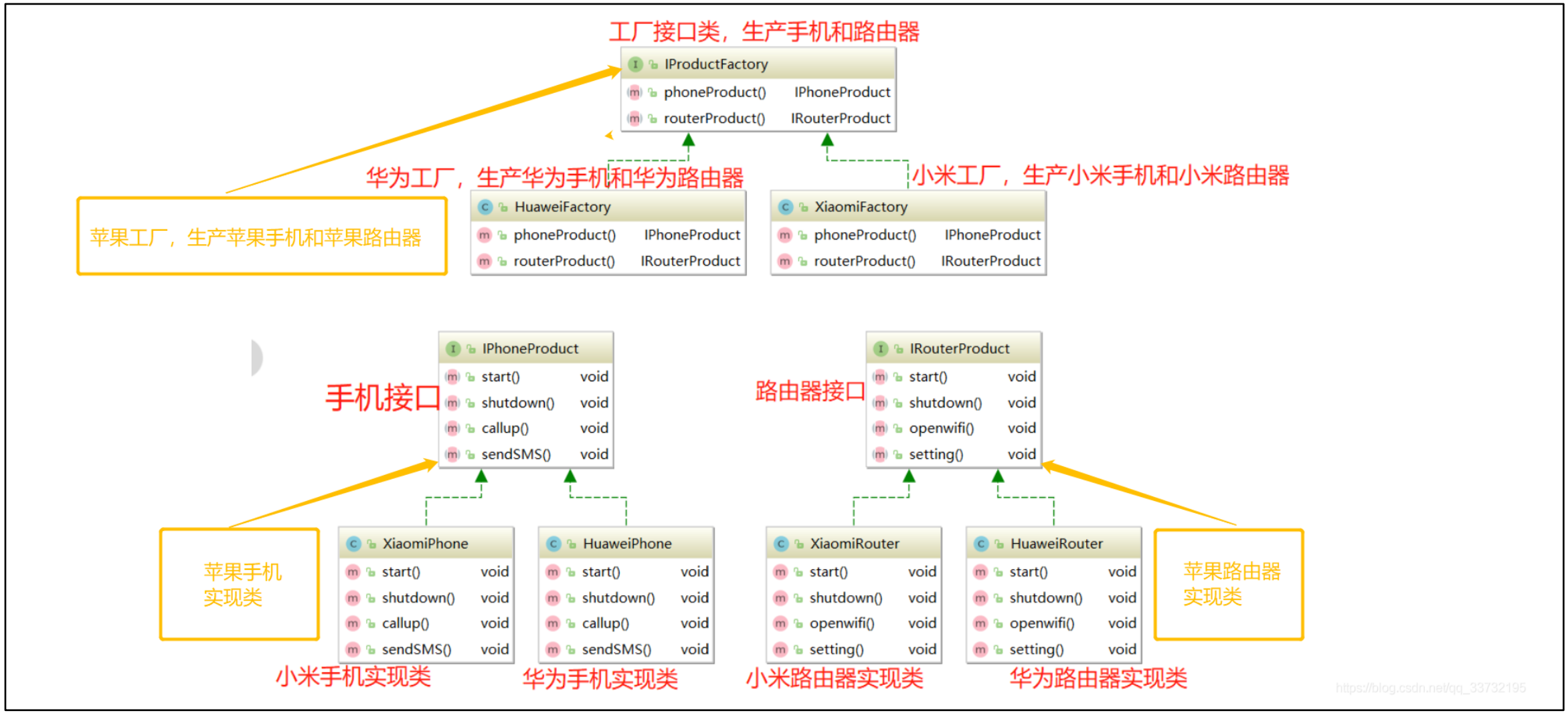

工厂接口类，生产手机和路由器
IProductFactory
phoneProduct() IPhoneProduct
routerProduct() IRouterProduct
集成到工厂接口类
华为工厂，生产华为手机和华为路由器
小米工厂，生产小米手机和小米路由器
HuaweiFactory
phoneProduct() IPhoneProduct
routerProduct() IRouterProduct
XiaomiFactory
phoneProduct() IPhoneProduct
routerProduct() IRouterProduct
对应的华为笔记本电脑实现方法
对应的小米笔记本电脑实现方法
笔记本电脑接口
手机接口
IPhoneProduct
start() void
shutdown() void
callup() void
sendSMS() void
路由器接口
IRouterProduct
start() void
shutdown() void
openwifi() void
setting() void
小米笔记本电脑实现类
华为笔记本电脑实现类
XiaomiPhone
HuaweiPhone
XiaomiRouter
HuaweiRouter
小米手机实现类
华为手机实现类
小米路由器实现类
华为路由器实现类

抽象工厂模式

拓展一个品牌（产品族）

- 如果新增一个品牌，也就是说新增一个厂商来生产手机和路由器，如下页图片所示（黄色字体为新增一个产品族需要做的事），新增一个产品族不用修改原来的代码，符合OCP原则，十分方便。

抽象工厂模式


工厂接口类，生产手机和路由器
IProductFactory
phoneProduct() IPhoneProduct
routerProduct() IRouterProduct
华为工厂，生产华为手机和华为路由器
小米工厂，生产小米手机和小米路由器
苹果工厂，生产苹果手机和苹果路由器
HuaweiFactory
phoneProduct() IPhoneProduct
routerProduct() IRouterProduct
XiaomiFactory
phoneProduct() IPhoneProduct
routerProduct() IRouterProduct
手机接口
IPhoneProduct
start() void
shutdown() void
callup() void
sendSMS() void
路由器接口
IRouterProduct
start() void
shutdown() void
openwifi() void
setting() void
苹果手机
实现类
XiaomiPhone
start() void
shutdown() void
callup() void
sendSMS() void
HuaweiPhone
start() void
shutdown() void
callup() void
sendSMS() void
XiaomiRouter
start() void
shutdown() void
openwifi() void
setting() void
HuaweiRouter
start() void
shutdown() void
openwifi() void
setting() void
苹果路由器
实现类
小米手机实现类
华为手机实现类
小米路由器实现类
华为路由器实现类
https://blog.csdn.net/qq_33732195

抽象工厂模式优缺点

- 优点
  - **产品族扩展非常容易**，只要新增具体工厂的实现和产品的实现。
  - 一个产品族中的多个对象被设计成一起工作时，它能保证客户端始终只使用同一个产品族中的对象（将一个系列的产品**统一**一起创建）。
- 缺点
  - **产品类型扩展非常困难**，要新增一个产品类，既要**修改**工厂抽象类里加代码，又修改具体的实现类里面加代码。
  - 增加了系统的抽象性和理解难度。